UNIT I – C++ DEBUGGING SOLUTIONS

Principles of Object-Oriented Programming & Control Structures

Solutions are based on the five debugging problems in the uploaded Unit I question bank. Each answer includes error analysis, corrected code, a brief dry run, and expected output.

Question 1: Operator Precedence and Unsigned Loop

Errors and root causes

- Line containing num & 1 == 1: equality (==) has higher precedence than bitwise AND (&). It is parsed as num & (1 == 1), which becomes num & 1; the result is an integer, not the intended Boolean comparison. Parenthesize the bitwise result: ((num & 1) == 1).
- The condition i >= 0 is always true for an unsigned counter. When i is 0, decrement wraps to the maximum value representable by unsigned char (typically 255), so the loop can continue indefinitely. Avoid an unsigned countdown condition of i >= 0.

Corrected C++ code

```
#include <iostream>

int main() {
    unsigned char num = 15;

    if (((num & 1) == 1) && num > 10) {
        std::cout << "Valid Odd Number: " << static_cast<int>(num) << '\n';
    } else {
        std::cout << "Condition Failed for: " << static_cast<int>(num) << '\n';
    }

    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << '\n';
    return 0;
}
```

Dry run and expected output

num = 15: 15 & 1 = 1, so the number is odd; 15 > 10 is true. The countdown prints 3, 2, 1, 0.

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution and Variable Shadowing

Errors and root causes

- The expression itemPrice * :taxRate is a syntax error. The global variable is accessed using the unary scope resolution operator as ::taxRate.

- ::taxRate refers to the global variable (0.18), not the enclosing local variable. The scope resolution operator cannot select an outer local variable by name. Rename the inner variable to prevent shadowing and keep the outer local variable accessible.

Corrected C++ code

```
#include <iostream>

double taxRate = 0.18;

int main() {
    double itemPrice = 1000.0;
    double subsidizedTaxRate = 0.05;

    std::cout << "Local Rate: " << subsidizedTaxRate << '\n';

    double totalWithGlobalTax = itemPrice + itemPrice * ::taxRate;

    {
        int corporateTaxRate = 12;
        double totalWithLocalTax =
            itemPrice + itemPrice * subsidizedTaxRate;
        std::cout << "Corporate Rate: " << corporateTaxRate << '\n';
        std::cout << "Inner total: " << totalWithLocalTax << '\n';
    }

    std::cout << "Final Global Calculation: "
              << totalWithGlobalTax << '\n';
    return 0;
}
```

Dry run and expected output

Local rate: 5% → 1000 + 50 = 1050. Global rate: 18% → 1000 + 180 = 1180. The nested corporate rate is 12 and is displayed separately.

```
Local Rate: 0.05
Corporate Rate: 12
Inner total: 1050
Final Global Calculation: 1180
```

Question 3: Switch Case Initialization and Case Types

Errors and root causes

- Declaring initialized variables directly across switch cases can cause a jump-over-initialization error: control may jump to another case past a variable initialization. Enclose each case body in braces to create its own scope.
- A switch expression must have an integral or enumeration type (after integral promotions); case labels must be converted constant expressions of a compatible integral/enum type. The floating-point label case 4.5 is not permitted.
- The missing break after case 2 causes fall-through into case 3, adding another 100 unintentionally.

Corrected C++ code

```
#include <iostream>
```

```
int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }
        case 3: {
            tokenValue += 100.0f;
            break;
        }
        case 4: {
            std::cout << "Float token detected" << '\n';
            break;
        }
        default:
            std::cout << "Unknown token" << '\n';
    }

    std::cout << "Calculated Value: " << tokenValue << '\n';
    return 0;
}
```

Dry run and expected output

tokenType = 2 selects case 2. multiplier = 5, so tokenValue = 20 × 5 = 100. The break prevents case 3 from executing.

```
Calculated Value: 100
```

Question 4: Reference Initialization and Const Binding

Errors and root causes

- int &refA; is invalid because a non-const lvalue reference must be initialized when declared. Bind it directly to a.
- int &refLiteral = 100; is invalid because a non-const lvalue reference cannot bind to a temporary literal. A const int& can bind to the temporary, extending its lifetime to the lifetime of the reference.
- int &refInt = pi; is invalid: pi is a double lvalue, while refInt is an int reference. A conversion would create a temporary int, and a non-const int& cannot bind to that temporary. Use an int variable explicitly initialized from pi.
- A const reference cannot be used to modify its referent. constRef = constRef + 10; is invalid. The reference can read the value but not assign through it.

Corrected C++ code

```
#include <iostream>

int main() {
```

```
    int a = 25;
    int &refA = a;

    const int &refLiteral = 100;

    double pi = 3.14159;
    int convertedPi = static_cast<int>(pi);
    int &refInt = convertedPi;

    const int &constRef = 50;

    std::cout << a << " " << refA << '\n';
    std::cout << refLiteral << " " << refInt << '\n';
    std::cout << constRef << '\n';
    return 0;
}
```

Dry run and expected output

refA aliases a, so both display 25. refLiteral reads 100. pi is explicitly converted to int, producing 3, and refInt aliases convertedPi. constRef reads 50 without modifying it.

```
25 25
100 3
50
```

Question 5: Dynamic Array and Stray Loop Semicolon

Errors and root causes

- The semicolon immediately after the for statement makes the loop body empty. The following block is separate from the loop, and i is out of scope there; using i in that block causes a compilation error.
- An array allocated with new[] must be released with delete[]. Using scalar delete for an array is undefined behavior and can cause memory-management errors.

Corrected C++ code

```
#include <iostream>

int main() {
    int size = 5;
    int *readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    std::cout << "Total: " << total << '\n';

    delete[] readings;
    return 0;
```

```
}
```

Dry run and expected output

The array is initialized to 10, 20, 30, 40, 50. Their sum is 10 + 20 + 30 + 40 + 50 = 150. The allocated array is released with delete[].

```
Total: 150
```

Quick Revision: Key C++ Rules

- Use parentheses when combining bitwise and comparison operators.
- Unsigned values cannot represent negative numbers; avoid i >= 0 countdown loops with unsigned counters.
- Use ::name to refer to a global variable; it does not access an outer local variable.
- Put declarations inside braces within switch cases; use integral case labels and prevent unintended fall-through with break.
- Initialize references immediately; const references can bind to temporaries but cannot modify them.
- Match new[] with delete[] and remove accidental semicolons after loop headers.